Evaluating Optimization Algorithms for Machine Learning

Abstract

Optimization algorithms are a cornerstone of machine learning, dictating how effectively models converge to solutions. Despite their success, many algorithms face significant limitations, such as sensitivity to hyperparameters, inefficiency on non-convex landscapes, and lack of adaptability across tasks. In this work, we evaluated existing optimization algorithms across diverse tasks in various domains to understand the conditions under which they perform optimally and explore enhancements like learning rate scheduling and warm-up strategies. By systematically applying and analyzing these algorithms, we provide insights into adapting optimization techniques to improve model performance across different scenarios. The implementation is available at https://bit.ly/optimizer_evaluation.

1. Introduction

Optimization algorithms are fundamental to machine learning, forming the backbone of model training by iteratively minimizing a loss function that quantifies prediction errors. These algorithms guide the adjustment of model parameters, ensuring that the model improves its predictions with each iteration. The effectiveness of an optimization algorithm directly impacts a model's ability to converge to an optimal or near-optimal solution, influencing both training speed and final performance.

Over the years, numerous optimization algorithms have been developed to address the multifaceted challenges of training machine learning models: scalability, efficiency, and adaptability (Ruder, 2016). By addressing these challenges, optimization algorithms have enabled significant advancements in the training of diverse machine learning models, from linear classifiers to deep neural networks (LeCun et al., 2015).

In this project, we aimed to systematically evaluate the effectiveness of several optimization algorithms across four diverse machine learning tasks: image classification, text classification, language modeling, and semantic segmentation. These tasks represent fundamental problems in the fields of computer vision and natural language processing, offering a comprehensive view of how optimization strategies perform under varying data modalities and model architectures.

We focused on the following widely-used optimization algorithms: Adam, AdamW, AdaGrad, RMSProp, and SGD. To better understand their impact, we investigated experiments using a baseline optimization, optimization with learning rate schedulers and optimization with warm-up techniques.

Learning rate scheduling refers to techniques that dynamically adjust the learning rate during the training process. This strategy is essential for improving convergence, avoiding overshooting the optimal solution, and fine-tuning the learning process. Common scheduling approaches include step-based decay, exponential decay, and polynomial decay.

Warm-up techniques gradually increase the learning rate from a small initial value to the target learning rate over a specified number of iterations or epochs. This method is particularly beneficial for stabilizing training in the early stages, especially when working with large datasets or complex models, such as transformers. By starting with a small learning rate, the model avoids large weight updates that can destabilize training, allowing it to "warm up" before settling into the standard optimization process.

Through this experiment, we aimed to identify patterns and best practices for optimizing performance in these tasks, as well as provide insights into the interplay between optimizers, warm-up techniques, and schedulers. By comparing results across multiple tasks and configurations, we expect to highlight the strengths and weaknesses of each optimizer and their adaptations, ultimately contributing to a deeper understanding of optimization in machine learning.

2. Related Works

Stochastic Gradient Descent (SGD) is one of the simplest and most widely used optimization algorithms in machine learning. Unlike standard gradient descent, which computes the gradient over the entire dataset, SGD uses a single data point or a small mini-batch for each update, introducing stochasticity into the optimization process. This characteristic makes SGD computationally efficient and adaptable to large-scale problems.

RMSProp (Root Mean Square Propagation) is an optimization algorithm designed to address challenges in train-

ing machine learning models, particularly when dealing with non-stationary objectives or sparse gradients. Developed as an adaptive learning rate method, RMSProp builds on the idea of maintaining a moving average of squared gradients to normalize parameter updates. This ensures that updates are scaled appropriately, regardless of the magnitude of gradients, which is especially beneficial for tasks involving complex or noisy loss landscapes.

Adam (Adaptive Moment Estimation)(Kingma & Ba, 2015) is one of the most widely used optimization algorithms in modern machine learning, combining the advantages of both RMSProp and Momentum. Proposed by Kingma and Ba (2015), Adam builds on the idea of maintaining adaptive learning rates for each parameter while incorporating the benefits of first-order momentum for smoother updates. Its ability to dynamically adjust learning rates during training makes Adam particularly effective for deep learning tasks and non-convex optimization problems .

AdaGrad (Adaptive Gradient Algorithm)(Duchi et al., 2011) is an adaptive learning rate optimization algorithm introduced to address challenges in training models on sparse data and features. Unlike standard gradient descent, which uses a fixed learning rate throughout training, AdaGrad dynamically adjusts the learning rate for each parameter based on the historical gradients observed during training. This feature makes AdaGrad particularly effective for tasks involving sparse gradients, such as natural language processing and recommendation systems.

AdamW (Adam with Weight Decay Regularization) (Loshchilov & Hutter, 2019) is an extension of the Adam optimization algorithm, specifically designed to address the shortcomings of Adam in scenarios requiring proper weight regularization. AdamW improves upon the Adam optimizer by decoupling weight decay from gradient updates, ensuring proper regularization by applying weight decay directly to the model's weights. This adjustment addresses Adam's suboptimal handling of weight decay, enhancing generalization performance and making AdamW particularly effective for training large-scale models in tasks like computer vision and natural language processing..

3. Experiment

3.1. Experiment Setup

We experimented with five optimization algorithms: Adam, AdamW, Adagrad, RMSProp, and SGD. Each optimizer was initialized with task-specific learning rates and weight decay values based on best practices in deep learning. For a comprehensive evaluation, we tested three different configurations for each optimizer:

1. Baseline optimization without additional enhancements.
2. Optimization with learning rate schedulers, to dynamically adjust the learning rate during training for better convergence.
3. Optimization with warm-up techniques, which are often used to stabilize training in the early stages.

These configurations allowed us to analyze the performance of each optimizer under varying conditions, providing insights into their effectiveness and adaptability across different training setups.

We employed two types of polynomial learning rate schedulers with power values of 0.9 and 1.0, respectively, and utilized a 1500-step warm-up phase with the poly10 schedule.

For the text classification task, we focused on evaluating optimization algorithms on the widely used AG News dataset, which consists of news articles classified into four categories. For the model and tokenizer, we used TinyBERT(Jiao et al., 2019) (4L, 312D) pre-trained model from HuggingFace for sequence classification, configured for four output classes. The corresponding tokenizer was used to preprocess the text data with truncation and padding to a maximum sequence length of 512 tokens. The model was trained for 3 epochs. For each batch, we calculated the cross-entropy loss between predicted logits and ground truth labels and updated model parameters using backpropagation.

For the language modeling task, we examined the performance of optimization algorithms on the widely recognized WikiText-2 dataset, a high-quality corpus commonly used for language modeling. We employed a custom LSTM-based language model(Hochreiter & Schmidhuber, 1997) with two layers, an embedding size of 128, and a hidden size of 256. The model, designed to predict the next token in a sequence, utilized a vocabulary size defined by the GPT-2 tokenizer. Input sequences were truncated to a maximum length of 35 tokens and grouped into batches of size 20 for efficient processing. The model was trained over 3 epochs. Cross-entropy loss was used to measure the difference between the predicted logits and the ground truth targets in each batch, with parameters updated through backpropagation. To maintain stability during training and prevent gradient explosion, the model's hidden states were appropriately detached or reset.

For the image classification task, we evaluated the performance of optimization algorithms on the widely used CIFAR-10 dataset, a benchmark dataset containing images classified into ten categories. We utilized the ResNet-18 architecture, modified for ten output classes by replacing the final fully connected layer. The input images were pre-

processed with data augmentation techniques, including random cropping, horizontal flipping, and normalization. The dataset was divided into training and test sets, with a batch size of 32. The model was trained for 20 epochs, where cross-entropy loss was computed between the predicted outputs and the ground truth labels for each batch. Backpropagation was used to update the model parameters, with learning rate adjustments applied through schedulers or warm-up techniques in specific configurations.

In the semantic segmentation task, we use deeplabV3 model, and backbone is Resnet-50 which is pretrained on ImageNet. And we apply random crop, random flip and photometric distortortion on the source images for data augmentation. We set the learning rate, momentum (if needed), and weight decay appropriate for each optimizer. For performance evaluation on different domains (GTA5 $\rightarrow$ Cityscapes), all optimizers were used with warm-up. We used 60,000 iterations, with validation evaluations conducted every 4,000 iterations. We utilized a total of two RTX 2080 Ti GPUs; however, the training was conducted on a single GPU. This task aims to evaluate the performance of each optimization method, compare training times, and assess generalization capabilities.

3.2. Dataset

For text classification, we use 20 Newsgroups dataset. The 20 Newsgroups dataset is a popular benchmark for text classification and clustering tasks, consisting of approximately 20,000 newsgroup documents organized into 20 categories such as technology, sports, politics, and science. Each document is labeled with one of the categories, making it suitable for multiclass classification problems. The dataset offers both raw and preprocessed versions, allowing for flexible use in experiments. Its diversity and inclusion of overlapping topics, such as "comp.sys.mac.hardware" and "comp.sys.ibm.pc.hardware," provide a challenging testbed for evaluating model accuracy and generalization.

For language modeling, the WikiText2 dataset is an ideal choice, derived from high-quality English Wikipedia articles. Known for its clean and coherent text, WikiText2 is designed to evaluate natural language understanding and text generation models. It includes longer contextual information, enabling models to capture dependencies over extended sequences. To assess the performance of the proposed optimization algorithm across different domains, we will include the AG News dataset. We will only use the subset of the AG News data for testing the language modeling.

To perform semantic segmentation, we use the Cityscapes dataset, a well-known road scene benchmark for semantic segmentation. This dataset consists of real urban street images from European cities and includes 2,975 training images and 500 validation images. As the test set does not provide ground truth, only the training and validation sets will be used. We will train the model to segment 19 semantic classes, including roads, pedestrians, vehicles, and buildings. To further assess the model's performance across different domains, we incorporated the GTA5 dataset, which also contains road scenes but consists of synthetic images. GTA5 provides a total of 24,966 images labeled with the same 19 classes as Cityscapes. Although widely used in Unsupervised Domain Adaptation (UDA) research by training on GTA5 and testing on Cityscapes, this project will not involve domain adaptation. Instead, we focus on evaluating whether the model can generalize across shifted domains with different distributions, assessing the optimizer's capability for robust performance in varied contexts.

For image classification tasks, the CIFAR-10 dataset will be utilized. CIFAR-10 includes 60,000 32x32 color images evenly distributed across 10 categories, such as airplanes, cats, and trucks, making it a compact yet diverse dataset for evaluating classification model performance.

Together, these datasets enable a comprehensive evaluation of the proposed optimizer across multiple tasks and domains.

3.3. Metrics

To evaluate the performance of the proposed optimization algorithms across different tasks, we will use task-specific and widely accepted evaluation metrics.

For semantic segmentation tasks, we will use Mean Intersection over Union (mIoU) as the primary evaluation metric:

$$\text{mIoU} = \frac{1}{n} \sum_{i=1}^{n} \frac{|P_i \cap G_i|}{|P_i \cup G_i|} \tag{1}$$

where P_i is the predicted segmentation for class i, G_i is the ground truth for class i, $|P_i \cap G_i|$ is the intersection, and $|P_i \cup G_i|$ is the union.

For language modeling tasks, we will use Perplexity to evaluate the quality of the predictions generated by the model:

$$\text{Perplexity} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log p(w_i)\right) \tag{2}$$

where N is the number of words in the sequence, and $p(w_i)$ is the predicted probability of word w_i. The term inside the parentheses, $-\frac{1}{N}\sum_{i=1}^{N} \log p(w_i)$, is equivalent to the cross-entropy loss. This measures how well the predicted probability distribution aligns with the true distribution. A lower value indicates better alignment between the predicted and true probability distributions.

For both image classification (CIFAR-10) and text classification (20 Newsgroups and AG News), we will evaluate

performance using accuracy, precision, recall, and F1-score.

4. Results

4.1. Results on Semantic Segmentation

4.1.1. Results on CityScapes

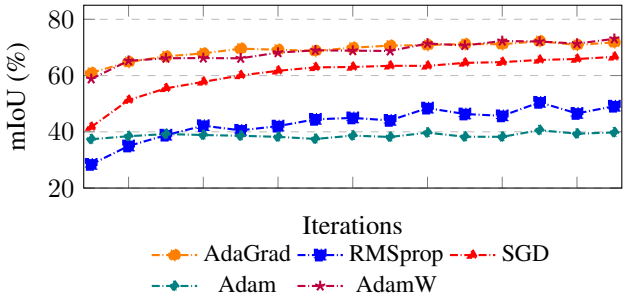

Figure 1. Validation mIoU Over Training Iterations for Different Optimizers

Figure 1 illustrates the validation mIoU across different optimizers over 60,000 iterations, evaluated every 4,000 iterations. The final accuracy rankings are as follows: AdamW, AdaGrad, SGD, RMSProp, and Adam.

AdamW and AdaGrad demonstrate rapid initial learning, achieving high performance early on. However, their overall performance improvement towards the end is relatively modest (15–20%). SGD, on the other hand, does not start as strong as AdaGrad or AdamW but converges steadily and shows stable performance across iterations. Unlike the others, SGD does not completely saturate, highlighting its potential for improved generalization due to its reliance on stable learning dynamics and slower saturation.

RMSProp begins with low performance and converges to a lower final accuracy compared to other optimizers. This result aligns with RMSProp's typical behavior of fast initial convergence but suboptimal long-term stability. Adam also demonstrates quick early updates but falls short in both convergence speed and final accuracy, likely due to insufficient optimization of the learning rate schedule or a tendency to overfit.

Optimizer	ori	polylr	poly10	LR Warm-up	Rel.
RMSProp	50.48	62.38	62.07	**66.09**	+30.92 %
SGD	**66.65**	64.01	63.73	63.88	-4.16 %
AdamW	73.06	73.99	**74.22**	73.81	+1.03 %
AdaGrad	**72.23**	71.27	70.79	70.86	-1.90 %
Adam	40.58	**45.89**	45.73	45.50	+12.12 %

Table 1. Comparison of optimizers with Schedulers, LR Warm-up and Relative Improvement

In Table 1, ori represents the original version without any schedulers or warm-up applied. polylr refers to the Polynomial Decay Schedule with a power of 0.9, while poly10 uses a power of 1.0. Rel indicates the relative improvement between the ori and warm-up results, expressed as a percentage.

The table 1 reveals some intriguing results regarding SGD and AdaGrad. For RMSProp, AdamW, and Adam, applying scheduling or warm-up led to improved performance compared to using no modifications, with RMSProp and Adam showing particularly significant improvements. In contrast, SGD and AdaGrad performed better without scheduling or warm-up. These differences are closely tied to the characteristics of each optimizer.

The ultimate purpose of learning rate scheduling and warm-up is to naturally reduce sharpness in the loss landscape, promoting stable training. Warm-up achieves this by gradually increasing the learning rate during the initial training phase, allowing the optimizer to navigate toward flatter regions. However, in the case of SGD, early-stage learning is crucial. SGD is designed to quickly move out of sharp regions in the loss landscape and into flat regions during the initial stages of training. Consequently, gradually increasing the learning rate via warm-up may hinder this early progress, potentially leading to a decrease in performance. This aligns with SGD's reliance on a large initial learning rate to effectively capture critical features during the early stages.

In our experiments, the learning rate and batch size were relatively small, which may have limited the effectiveness of warm-up. However, since RMSProp, AdamW, and Adam showed performance improvements with warm-up under the same experimental conditions, the aforementioned reasons for SGD's performance degradation seem more plausible.

For AdaGrad, it employs a mechanism where the learning rate is automatically reduced based on the squared sum of gradients for each parameter batch. Due to this characteristic, AdaGrad inherently includes a scheduling effect. It begins with a large learning rate and gradually adjusts it according to the magnitude of the gradients. However, introducing warm-up causes the initial learning rate to increase slowly, potentially conflicting with AdaGrad's natural learning rate adjustment mechanism. Similar to SGD, this could delay or impede effective early learning, leading to reduced performance.

In contrast, warm-up and scheduling positively impacted the performance of RMSProp. RMSProp adjusts the learning rate based on the root mean square of the gradients, enabling stable training even during the highly dynamic early stages. Warm-up complements RMSProp's strengths by ensuring more stable progress during initial training.

Additionally, Adam combines the first moment (gradient

mean) and the second moment (gradient squared mean) to adjust the learning rate. Warm-up prevents excessive gradient spikes (loss catapults) in the early stages and provides time for the second moment to stabilize, significantly enhancing training stability. Adam is particularly sensitive to learning rate fluctuations when using small batch sizes, making warm-up especially effective in stabilizing early training.

In conclusion, for SGD and AdaGrad, the initial learning characteristics and inherent mechanisms of these optimizers may have caused scheduling and warm-up to result in reduced performance. On the other hand, for RMSProp, Adam, and AdamW, warm-up and scheduling improved early training stability and mitigated gradient fluctuations, contributing to their enhanced performance.

Optimizer	Training time
RMSProp	9h 41m
SGD	9h 17m
AdamW	9h 47m
AdaGrad	9h 18m
Adam	9h 41m

Table 2. Comparison of optimizers with Training time

Table 2 presents the training time for each optimizer, emphasizing efficiency alongside final performance. The results reflect training conducted without scheduling or warm-up. The training time was measured to assess how computationally demanding each optimizer is, considering their different algorithms and internal adjustments.

Among the optimizers, AdamW recorded the longest training time at 9 hours and 47 minutes. This is likely due to its use of weight decay instead of L2 regularization and its reliance on more complex internal learning rate adjustments. In contrast, SGD showed the shortest training time at 9 hours and 17 minutes, reflecting its relatively simple update rule, which involves fewer computational overheads.

Other optimizers, such as RMSProp and AdaGrad, demonstrated intermediate training times, as their dynamic learning rate adjustments and accumulation mechanisms require additional computation but are less demanding than AdamW. Adam shared similar training time characteristics with RMSProp due to its adaptive moment estimation approach.

4.1.2. Generalization Performance

Figure 2 shows the results of training on the GTA5 dataset and testing on Cityscapes, with validation performed every 4000 iterations across a total of 60,000 iterations, similar to Figure 1. To enhance generalization performance, warm-up was applied to all optimizers(Hoyer et al., 2022).

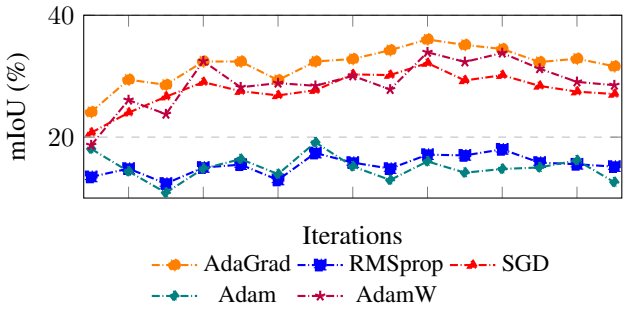

Figure 2. Validation mIoU Over Training Iterations for Different Optimizers on Other Domain

This experiment examines which optimizer performs best when a domain distribution shift occurs. Ultimately, AdaGrad demonstrated the best performance, confirming that AdaGrad, AdamW, and SGD are optimizers with superior generalization capabilities.

The strong generalization performance of these three optimizers can be attributed to their inherent mechanisms:

- AdaGrad excels in adapting to domain shifts by adjusting learning rates based on gradient accumulation, enabling it to effectively learn rare features in the new domain.
- AdamW mitigates overfitting through weight decay, providing stable learning even under significant domain distribution changes.
- SGD, with its simplicity and robust optimization, gradually converges toward a more global minimum, showing potential for further improvement with extended training.

As mentioned earlier, SGD has not fully converged within 60,000 iterations, suggesting that its generalization performance could improve further with additional training.

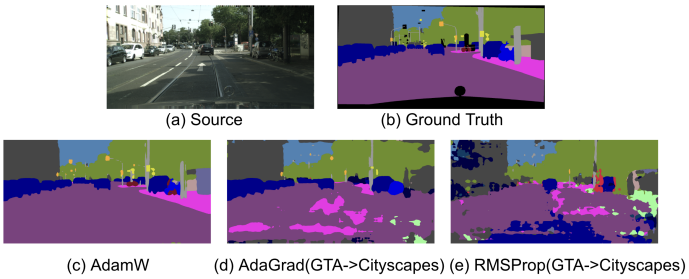

Figure 3. Semantic Segmentation Map Result

In Figure 3, (c) illustrates the results of AdamW, showing the highest accuracy in same-domain testing. AdamW

aligns most closely with the Ground Truth in the source domain, while (d) and (e) present cross-domain evaluation results (GTA5 → Cityscapes). A comparison between AdaGrad, which achieved the highest generalization performance, and RMSProp, which had the lowest, reveals that AdaGrad maintains closer alignment with the Ground Truth in the target domain, demonstrating superior generalization capability.

4.1.3. Ranking on Semantic Segmentation Task

Optimizer	Performance	Time Consumption	Generalization	Total Rank
RMSProp	4	3	5	4
SGD	3	**1**	3	3
AdamW	**1**	5	2	2
AdaGrad	2	2	**1**	**1**
Adam	5	3	4	5

Table 3. Ranking of Optimization on Semantic Segementation Task

Table 3 presents the ranking of optimizers on the semantic segmentation task. The rankings were determined using weighted scores, with performance given a weight of 0.5, generalization 0.4, and time consumption 0.1. Performance rankings were based on the highest mIoU achieved, including scheduling and warm-up. As a result, AdaGrad secured the top position, followed by AdamW and SGD.

4.2. Results on Language Modeling

Optimizer	Configuration	Test Perplexity
AdaGrad	Warm-Up	2.7158
	Scheduler	**2.6999**
	No Scheduler, No Warm-Up	2.7410
Adam	Warm-Up	3.1245
	Scheduler	**3.1054**
	No Scheduler, No Warm-Up	3.1352
AdamW	Warm-Up	2.5984
	Scheduler	**2.5910**
	No Scheduler, No Warm-Up	2.6023
RMSProp	Warm-Up	2.8100
	Scheduler	**2.7980**
	No Scheduler, No Warm-Up	2.8250
SGD	Warm-Up	3.4567
	Scheduler	**3.4410**
	No Scheduler, No Warm-Up	3.4789

Table 4. Test Metrics Comparison on language modeling

Based on the table 4, applying scheduler and warm-up consistently led to better performance across all optimizers compared to the default configuration of "No Scheduler, No Warm-Up." This indicates that learning rate adjustment techniques play a significant role in stabilizing training and improving test perplexity.

This consistent improvement can be attributed to the ability of learning rate adjustment techniques, like scheduler and warm-up to provide a smoother and more stable optimization process, especially during the early stages of training. In language modeling tasks, the loss landscape is often complex and highly non-convex, with sharp gradients that can lead to instability or suboptimal convergence if the learning rate is not carefully managed. Warm-Up helps by gradually increasing the learning rate from a small initial value, preventing abrupt gradient fluctuations and allowing the optimizer to navigate the loss landscape more effectively. Similarly, schedulers dynamically adjust the learning rate throughout training, helping to maintain stability and ensuring that the optimizer does not overshoot or converge prematurely, ultimately leading to improved generalization and lower test perplexity.

The best result in the language modeling experiment was achieved with AdamW, which recorded the lowest test perplexity when both scheduler and warm-up were applied. AdamW combines the advantages of Adam's adaptive optimization with weight decay regularization, which helps prevent overfitting and promotes better generalization. In language modeling, where the model must capture complex dependencies and patterns across sequences, AdamW's adaptive learning rate mechanism efficiently adjusts to the varying gradient magnitudes across parameters. The application of warm-up further stabilizes the optimization process by preventing large updates in the early stages, while scheduler ensures gradual and controlled learning rate decay, allowing the model to fine-tune its parameters effectively as training progresses. This combination of adaptive optimization, regularization, and learning rate control makes AdamW particularly well-suited for language modeling tasks, resulting in superior performance compared to other optimizers.

One important consideration is that the optimal learning rate for each optimizer and configuration was not exhaustively tuned during these experiments. The observed improvements with scheduler and warm-up might have been influenced by the default learning rates chosen for the experiments. Warm-up, in particular, may not have shown its full potential if the initial learning rates were not precisely tuned for the optimizer-task combination. This highlights the possibility that better results might be achieved with more thorough exploration of learning rate settings, as the interplay between the initial learning rate and the warm-up phase can significantly affect training stability and convergence in language modeling.

Optimizer	Configuration	Test Accuracy	Test Precision
AdaGrad	Warm-up	0.9013	0.9012
	Scheduler	**0.9039**	**0.9043**
	No Scheduler No Warm-up	0.8724	0.8719
Adam	Warm-up	0.9321	0.9326
	Scheduler	**0.9368**	**0.9369**
	No Scheduler No Warm-up	0.9328	0.9334
AdamW	Warm-up	**0.9372**	**0.9384**
	Scheduler	0.9364	0.9367
	No Scheduler No Warm-up	0.9343	0.9349
RMSProp	Warm-up	**0.8882**	**0.8902**
	Scheduler	0.7503	0.7557
	No Scheduler No Warm-up	0.8699	0.8822
SGD	Warm-up	**0.8979**	**0.8977**
	Scheduler	0.8807	0.8806
	No Scheduler No Warm-up	0.8954	0.8962

Table 5. Test Metrics Comparison on Text Classification: Test Accuracy and Test Precision

Optimizer	Configuration	Test Recall	Test F1
AdaGrad	Warm-up	0.9013	0.9012
	Scheduler	**0.9039**	**0.9039**
	No Scheduler No Warm-up	0.8724	0.8717
Adam	Warm-up	0.9321	0.9321
	Scheduler	**0.9368**	**0.9368**
	No Scheduler No Warm-up	0.9328	0.9326
AdamW	Warm-up	**0.9372**	**0.9372**
	Scheduler	0.9364	0.9365
	No Scheduler No Warm-up	0.9343	0.9345
RMSProp	Warm-up	**0.8882**	**0.8876**
	Scheduler	0.7503	0.7489
	No Scheduler No Warm-up	0.8699	0.8717
SGD	Warm-up	**0.8979**	**0.8978**
	Scheduler	0.8807	0.8805
	No Scheduler No Warm-up	0.8954	0.8949

Table 6. Test Metrics Comparison on Text Classification: Test Recall and Test F1

4.3. Results on Text Classification

In Table 5 and 6, the result shows that applying scheduler and warm-up consistently led to improved performance across most optimizers in text classification tasks.

In text classification, the model must learn to identify patterns in textual data that are often highly contextual and feature-dependent. This process requires a stable optimization process to avoid issues such as overfitting or divergence. The results in the tables show that using scheduler and warm-up generally improved metrics such as test accuracy, precision, recall, and F1 score. These learning rate adjustment techniques help the optimizer traverse the loss landscape more effectively, especially during the initial stages of training, by preventing large gradient fluctuations and ensuring stable parameter updates.

The best overall performance across all metrics was achieved by AdamW with warm-up, which recorded the highest test accuracy, precision, recall, and F1 score. AdamW's superior performance can be attributed to its combination of adaptive learning rate adjustments (like Adam) and weight decay regularization. In text classification, where overfitting is a common challenge due to the high dimensionality of textual features, the weight decay in AdamW helps prevent overfitting by penalizing large weights. Additionally, "Warm-Up" helped mitigate the risk of unstable gradient updates early in training, a critical phase for learning meaningful embeddings in text classification tasks.

The results highlight the importance of using learning rate adjustment techniques, where stable optimization is crucial for capturing intricate patterns in textual data.

4.4. Results on Image Classification

Optimizer	Configuration	Test Precision	Test Accuracy
Adagrad	Warm-up	0.7837	0.7846
	Schedule	0.7774	0.7776
	No Scheduler No Warm-up	**0.7859**	**0.7870**
Adam	Warm-up	**0.7351**	0.7322
	Schedule	0.7122	0.7132
	No Scheduler No Warm-up	0.7264	**0.7217**
Adamw	Warm-up	0.8106	0.8081
	Schedule	**0.8197**	**0.8178**
	No Scheduler No Warm-up	0.8160	0.8140
Rmsprop	Warm-up	0.7827	0.7768
	Schedule	**0.8180**	**0.8190**
	No Scheduler No Warm-up	0.7863	0.7779
Sgd	Warm-up	0.7444	0.7369
	Schedule	0.7359	0.7371
	No Scheduler No Warm-up	**0.7578**	**0.7556**

Table 7. Test Metrics Comparison: Test Precision and Test Accuracy

In image classification tasks, optimizers play a critical role in navigating complex loss landscapes where gradients can vary significantly depending on the depth of the model and the dataset. From the results in the tables, "No Scheduler, No Warm-Up" consistently outperformed other configurations for most optimizers in terms of test precision, accuracy, recall, and F1 score. This result indicates that in image classification, where convolutional neural networks (CNNs) are typically used, the default learning rate setting may allow the model to converge effectively without requiring additional learning rate adjustments such as Warm-Up or Scheduler.

Unlike text classification, image classification involves spatial features that are less prone to the sharp gradient fluctuations seen in sequence data, which may explain why the impact of Warm-Up and Scheduler is less pronounced

Optimizer	Configuration	Test Recall	Test F1
Adagrad	Warm-up	0.7846	0.7830
	Schedule	0.7776	0.7769
	No Scheduler No Warm-up	**0.7870**	**0.7843**
Adam	Warm-up	0.7322	0.7319
	Schedule	0.7132	0.7123
	No Scheduler No Warm-up	**0.7217**	**0.7229**
Adamw	Warm-up	0.8081	0.8087
	Schedule	**0.8178**	**0.8180**
	No Scheduler No Warm-up	0.8140	0.8142
Rmsprop	Warm-up	0.7768	0.7745
	Schedule	**0.8190**	**0.8181**
	No Scheduler No Warm-up	0.7779	0.7802
Sgd	Warm-up	0.7369	0.7368
	Schedule	0.7371	0.7357
	No Scheduler No Warm-up	**0.7556**	**0.7552**

Table 8. Test Metrics Comparison: Test Recall and Test F1

here. Additionally, the regularity of image data enables optimizers to achieve better generalization even without gradual learning rate adjustmen

The best overall result in the image classification task was achieved with AdamW using Scheduler. AdamW's superior performance in this configuration can be attributed to its combination of adaptive learning rate adjustments and weight decay regularization, which work synergistically to prevent overfitting and improve generalization.

The use of Scheduler further enhanced AdamW's performance by dynamically reducing the learning rate as training progressed, allowing the model to fine-tune its parameters more effectively for the complex decision boundaries in image classification. This gradual adjustment helped AdamW avoid the risk of overshooting or converging to suboptimal solutions, which is especially important in tasks involving deep convolutional networks. The weight decay mechanism provided an additional layer of stability by discouraging large parameter values, ensuring a well-regularized model capable of robust performance on test data.

AdaGrad and SGD performed best in the "No Scheduler, No Warm-Up" configuration due to their inherent optimization characteristics. AdaGrad naturally adjusts learning rates based on the accumulation of squared gradients, making external scheduling redundant and potentially conflicting with its internal mechanism. SGD, on the other hand, benefits from a large, consistent initial learning rate, which allows it to quickly optimize spatial patterns in image classification tasks. Warm-Up or Scheduler can slow down or interfere with their strengths, resulting in suboptimal performance. Therefore, both optimizers achieve better results when allowed to rely on their default behavior without additional adjustments.

In conclusion, the results in Table 7 and 8 highlight the importance of understanding the inherent mechanisms of different optimizers. For SGD and AdaGrad, the initial learning characteristics made warm-up and scheduling less effective. Conversely, RMSProp, Adam, and AdamW benefited significantly from these techniques, which improved training stability and mitigated gradient fluctuations, leading to enhanced performance.

5. limitation

One of the primary limitations of this experiment is the lack of extensive hyperparameter tuning due to time constraints. Ideally, each optimization algorithm should be rigorously tested with various learning rates, weight decay values, and other hyperparameters to identify the optimal configuration for each specific task. However, given the limited timeframe, we relied on widely accepted default values and well-known best practices for these settings. While these defaults are generally effective, they may not represent the true potential of each algorithm in our specific experimental context. This limitation leaves room for further exploration to determine whether task-specific fine-tuning could lead to better performance outcomes.

6. conclusion

In conclusion, the evaluation of optimization algorithms across diverse machine learning tasks, including text classification, language modeling, image classification, and semantic segmentation, underscores the importance of aligning optimization techniques with the specific characteristics of each task. Advanced methods like learning rate schedulers and warm-up consistently improved performance for most optimizers. For text and language tasks, schedulers and warm-up proved critical in stabilizing training on highly non-convex loss landscapes, while for image classification, the benefits were more task-specific, with AdamW and RMSProp excelling under these configurations.

Conversely, AdaGrad and SGD performed best with "No Scheduler, No Warm-Up" in tasks like image classification and semantic segmentation, where their inherent mechanisms, such as natural learning rate adjustments (AdaGrad) and reliance on large initial learning rates (SGD), allowed for effective convergence without external modifications. These findings highlight the necessity of understanding the intrinsic properties of each optimizer and tailoring configurations—such as learning rates, weight decay, and scheduling techniques—to the unique challenges and data structures of each task. Future experiments focusing on comprehensive hyperparameter tuning and exploring hybrid or emerging optimization strategies could provide deeper insights and further enhance model performance across diverse domains.

References

Duchi, J., Hazan, E., and Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

Hoyer, L., Dai, D., and Van Gool, L. Daformer: Improving network architectures and training strategies for domain-adaptive semantic segmentation. *arXiv preprint arXiv:2111.14887*, 2022.

Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., Wang, F., and Liu, Q. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351*, 2019.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *Nature*, 521(7553):436–444, 2015.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.

Ruder, S. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.